

Nama : Irvan Cahya Nugraha

NIM : 245410034

Kelas : IF – 1

Rangkuman praktikum system terdesentralisasi dan terdistribusi

IMPEMENTASI PUB/SUB (BROADCASTING)

```
import pika

# --- KONFIGURASI ---
# Masukkan Private IP VM-1 (Server) Anda
HOST_BROKER = '172.31.67.195'
credentials = pika.PlainCredentials('admin', 'admin123')

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host=HOST_BROKER, port=5672, virtual_host='/', credentials=credentials))
channel = connection.channel()

# 1. Buat EXCHANGE tipe 'fanout'
# Nama dengan 'sistem_transaksi'
channel.exchange_declare(exchange='sistem_transaksi', exchange_type='fanout')

# 2. Buat temporary queue (entrian sementara)
# exclusive=True artinya jika script dimatikan, entrian hilang (bersih)
result = channel.queue_declare(queue='', exclusive=True)
nama_queue = result.method.queue

# 3. BINDING: Sambungkan entrian temporary ini ke Exchange
channel.queue_bind(exchange='sistem_transaksi', queue=nama_queue)

print(f' [*] SISTEM LOGGER siap. Menunggu data transaksi...')

def callback(ch, method, properties, body):
    print(f' [LOG] Menatat transaksi: {body.decode()}')

channel.basic_consume(queue=nama_queue, on_message_callback=callback, auto_ack=True)
channel.start_consuming()
```

```
import pika

HOST_BROKER = '172.31.67.195' # Masukkan IP
credentials = pika.PlainCredentials('admin', 'admin123')

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host=HOST_BROKER, port=5672, virtual_host='/', credentials=credentials))
channel = connection.channel()

# 1. Declare Exchange yang sama
channel.exchange_declare(exchange='sistem_transaksi', exchange_type='fanout')

# 2. Buat Temporary Queue
result = channel.queue_declare(queue='', exclusive=True)
nama_queue = result.method.queue

# 3. Binding ke Exchange yang sama
channel.queue_bind(exchange='sistem_transaksi', queue=nama_queue)

print(f' [*] SISTEM NOTIFIKASI siap. Menunggu alert...')

def callback(ch, method, properties, body):
    # Simulasi Kirim notif
    print(f' [ALERT] Mengirim SMS ke Admin: Ada {body.decode()}')

channel.basic_consume(queue=nama_queue, on_message_callback=callback, auto_ack=True)
channel.start_consuming()
```

```
import datetime

HOST_BROKER = '172.31.67.195' # Masukkan IP
credentials = pika.PlainCredentials('admin', 'admin123')

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host=HOST_BROKER, port=5672, virtual_host='/', credentials=credentials))
channel = connection.channel()

# Declare Exchange (agar aman jika publisher jalan duluan)
channel.exchange_declare(exchange='sistem_transaksi', exchange_type='fanout')

while True:
    barang = input("Masukkan nama barang terjual (atau 'exit'): ")
    if barang == 'exit':
        break

    waktu = datetime.datetime.now().strftime("%H:%M:%S")
    pesan = f"Pembelian {barang} pada jam {waktu}"

    # PUBLISH KE EXCHANGE (Bukan ke Queue!)
    # routing key dikosongkan karena fanout menyebar ke semua
    channel.basic_publish(exchange='sistem_transaksi', routing_key='', body=pesan)

    print(f' [x] Data terkirim ke sentral: {barang}')

connection.close()
```

```
Last login: Thu Nov 27 08:39:57 2025 from 103.98.239.23
ubuntu@ip-172-31-24-75:~$ nano sub_logger.py
ubuntu@ip-172-31-24-75:~$ nano sub_notif.py
ubuntu@ip-172-31-24-75:~$ nano sub_logger.py
ubuntu@ip-172-31-24-75:~$ nano sub_notif.py
ubuntu@ip-172-31-24-75:~$
```

Pembahasan : Masih menggunakan rabbitMq lanjutan praktik 8 kemarin kita perlu emnambah 3 script python yaitu subscriber/ penerima (sub_logger.py), Script Notifikasi (sub_notif.py) lalu Publisher/Pengirim (pub_transaksi.py)

```
ubuntu@ip-172-31-24-75:~$ python3 pub_transaksi.py
Masukkan nama barang terjual (atau 'exit'): laptop gaming
[x] Data terkirim ke sentral: 'laptop gaming'
Masukkan nama barang terjual (atau 'exit'): mouse wireless
[x] Data terkirim ke sentral: 'mouse wireless'
Masukkan nama barang terjual (atau 'exit'): 
```

i-000a8ff1c561f54d3 (Vm-client)

PublicIPs: 44.210.41.98 PrivateIPs: 172.31.24.75

```
Last login: Thu Dec  4 07:24:20 2025 from 18.206.107.27
ubuntu@ip-172-31-24-75:~$ python3 sub_notif.py
[*] SISTEM NOTIFIKASI siap. Menunggu alert...
[ALERT] Mengirim SMS ke Admin: Ada Pembelian laptop gaming pada jam 07:47:14
[ALERT] Mengirim SMS ke Admin: Ada Pembelian mouse wireless pada jam 07:47:22

```

i-000a8ff1c561f54d3 (Vm-client)

PublicIPs: 44.210.41.98 PrivateIPs: 172.31.24.75

```
Last login: Thu Dec  4 07:39:34 2025 from 18.206.107.28
ubuntu@ip-172-31-24-75:~$ python3 sub_logger.py
[*] SISTEM LOGGER siap. Menunggu data transaksi...
[LOG] Mencatat transaksi: Pembelian laptop gaming pada jam 07:47:14
[LOG] Mencatat transaksi: Pembelian mouse wireless pada jam 07:47:22

```

i-000a8ff1c561f54d3 (Vm-client)

PublicIPs: 44.210.41.98 PrivateIPs: 172.31.24.75

Pembahasan : berikutnya adalah uji coba 3 jendela untuk masing masing script yaitu logger python3 sub_logger.py, notif python3 sub_notif.py, publisher python3 pub_transaksi.py khusus pada publisher ktia akan menginput suatu benda yang terbeli jika diketik laptop gaming maka notif akan mengirim pesan ke admin dan logger akan mencatat keduanya akan otomatis berjalan ketika publisher menerima input

Tugas

```
GNU nano 7.2                                sub_logger_p.py
import pika

HOST_BROKER = '172.31.67.195' # Ganti dengan IP RabbitMQ
credentials = pika.PlainCredentials('admin', 'admin123')

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host=HOST_BROKER, port=5672,
        virtual_host='/', credentials=credentials))
channel = connection.channel()

# 1. Buat exchange baru untuk sistem pembayaran
channel.exchange_declare(exchange='sistem_pembayaran', exchange_type='fanout')

# 2. Buat temporary queue
result = channel.queue_declare(queue='', exclusive=True)
nama_queue = result.method.queue

# 3. Binding ke exchange
channel.queue_bind(exchange='sistem_pembayaran', queue=nama_queue)

print(f"[*] LOGGER PEMBAYARAN siap. Menunggu data pembayaran...")

def callback(ch, method, properties, body):
    print(f"[LOG PEMBAYARAN] Mencatat: {body.decode()}")

channel.basic_consume(queue=nama_queue, on_message_callback=callback, auto_ack=True)
channel.start_consuming()
```

[Read 27 lines]

^G Help

^O Write Out

^W Where Is

^K Cut

^T Execute

^C Location

M-U Undo

^X Exit

^R Read File

^_ Replace

^U Paste

^J Justify

^_ Go To Line

M-E Redo

i-000a8ff1c561f54d3 (Vm-client)

PublicIPs: 44.210.41.98 PrivateIPs: 172.31.24.75

```
GNU nano 7.2                                sub_notif_p.py
import pika

HOST_BROKER = '172.31.67.195'
credentials = pika.PlainCredentials('admin', 'admin123')

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host=HOST_BROKER, port=5672,
        virtual_host='/', credentials=credentials))
channel = connection.channel()

channel.exchange_declare(exchange='sistem_pembayaran', exchange_type='fanout')

result = channel.queue_declare(queue='', exclusive=True)
nama_queue = result.method.queue

channel.queue_bind(exchange='sistem_pembayaran', queue=nama_queue)

print(f"[*] NOTIFIKASI PEMBAYARAN siap. Menunggu alert...")

def callback(ch, method, properties, body):
    print(f"[ALERT PEMBAYARAN] SMS ke Admin: {body.decode()}")

channel.basic_consume(queue=nama_queue, on_message_callback=callback, auto_ack=True)
channel.start_consuming()
```

[Read 24 lines]

^G Help	^O Write Out	^W Where Is	^R Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify	^/ Go To Line	M-E Redo

i-000a8ff1c561f54d3 (Vm-client)

PublicIPs: 44.210.41.98 PrivateIPs: 172.31.24.75

```
GNU nano 7.2                                pub_p.py
import pika
import datetime

HOST_BROKER = '172.31.67.195'
CREDENTIALS = pika.PlainCredentials('admin', 'admin123')

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host=HOST_BROKER, port=5672,
        virtual_host='/', credentials=CREDENTIALS))
channel = connection.channel()

# Declare exchange yang sama
channel.exchange_declare(exchange='sistem_pembayaran', exchange_type='fanout')

while True:
    metode = input("Metode pembayaran (contoh: BNI, Mandiri, OVO): ")
    if metode.lower() == 'exit':
        break
    jumlah = input("Jumlah pembayaran (contoh: Rp10.000): ")

    waktu = datetime.datetime.now().strftime("%H:%M:%S")
    pesan = f"Pembayaran Diterima via {metode} sebesar {jumlah} pada {waktu}"

    channel.basic_publish(exchange='sistem_pembayaran', routing_key='', body=pesan)

    print(f"[x] Data pembayaran terkirim: '{pesan}'")

connection.close()
```

[Read 28 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

i-000a8ff1c561f54d3 (Vm-client)

PublicIPs: 44.210.41.98 PrivateIPs: 172.31.24.75

```
ubuntu@ip-172-31-24-75:~$ python3 pub_notif_p.py
python3: can't open file '/home/ubuntu/pub_notif_p.py': [Errno 2] No such file or directory
ubuntu@ip-172-31-24-75:~$ python3 pub_p.py
Metode pembayaran (contoh: BNI, Mandiri, OVO): BNI
Jumlah pembayaran (contoh: Rp10.000): 10.000
[x] Data pembayaran terkirim: 'Pembayaran Diterima via BNI sebesar 10.000 pada 08:07:32'
Metode pembayaran (contoh: BNI, Mandiri, OVO):
```

i-000a8ff1c561f54d3 (Vm-client)

PublicIPs: 44.210.41.98 PrivateIPs: 172.31.24.75

```
Last login: Thu Dec  4 07:39:35 2025 from 18.206.107.29
ubuntu@ip-172-31-24-75:~$ python3 sub_notif_p.py
[*] NOTIFIKASI PEMBAYARAN siap. Menunggu alert...
[ALERT PEMBAYARAN] SMS ke Admin: Pembayaran Diterima via BNI sebesar 10.000 pada 08:07:32
```

i-000a8ff1c561f54d3 (Vm-client)

PublicIPs: 44.210.41.98 PrivateIPs: 172.31.24.75

```
Last login: Thu Dec  4 07:59:58 2025 from 18.206.107.27
ubuntu@ip-172-31-24-75:~$ python3 sub_logger_p.py
[*] LOGGER PEMBAYARAN siap. Menunggu data pembayaran...
[LOG PEMBAYARAN] Mencatat: Pembayaran Diterima via BNI sebesar 10.000 pada 08:07:32
```

i-000a8ff1c561f54d3 (Vm-client)

PublicIPs: 44.210.41.98 PrivateIPs: 172.31.24.75

Pembahasan : Jadi untuk membuat pesan “Pembayaran Diterima via BNI sebesar Rp10.000” kita perlu memodifikasi topic didalam sistem karena sebelumnya kita membuat sistem pembelian kita ubah menjadi sistem keuangan dengan hanya mengganti script di python mengganti beberapa teks agar terlihat seperti meminta inputan nominal dan teks notif juga diubah agar sesuai

Kesimpulan : Pub sub Adalah semua subscriber atau penerima yang mengikuti saluran atau system akan menerima pesan dari publisher secara otomatis kecuali jika penerima sudah tidak berlangganan

